# Coral Pipeline — Onboarding Guide

2026-09-18 · @Someone

## Project overview

We're building an **automated coral identification pipeline** for reef monitoring in the Gulf of Eilat. The goal: take underwater photographs of coral reefs, identify every individual coral colony in each photo, and classify it to the genus level — producing structured data that ecologists can use to track reef health over time.

The pipeline has two main stages, each handled by its own notebook:

1. **Segmentation** (`CoralMasking.ipynb`) — An AI model scans each reef photo and draws a precise outline around every coral colony it finds. Each outline (called a *mask*) isolates one coral from the background and from its neighbors. The model also extracts visual features from each coral crop for later use.
2. **Classification** (`WeakClassification.ipynb`) — Each isolated coral crop is classified to one of 20 target hard coral genera using zero-shot classification. CLIP-based vision-language models compare each coral crop against text descriptions of genera — no coral-specific training data needed.

The end-to-end flow looks like this:

**Reef photo → SAM segmentation → individual coral masks → crop extraction → CLIP zero-shot classification → genus labels**

Everything runs in Google Colab with data stored on Google Drive. The two notebooks above are the ones you'll work with directly.

## Key concepts for image processing newcomers

### Image segmentation and masks

In ML you're used to classification ("this image is a cat") or detection ("there's a cat at these coordinates"). **Segmentation** goes further: it labels *every pixel* in an image as belonging to a specific object or background. The output is a **mask** — a binary image the same size as the original where 1 = "this pixel belongs to the object" and 0 = background.

For coral reefs, one photo might contain dozens of overlapping colonies. Segmentation gives us a separate mask for each one, so we can crop out individual corals for classification.

## SAM (Segment Anything Model)

**SAM** is Meta/Facebook's foundation model for image segmentation, trained on 11 million images. It's designed to segment *any* object in *any* image without knowing what that object is. Think of it as a general-purpose "draw an outline around each thing" model.

Our pipeline uses **CoralSCOP**, a version of SAM (ViT-B architecture) that was fine-tuned specifically on coral reef imagery. This fine-tuning means it understands coral colony boundaries better than generic SAM — it knows where one coral ends and another begins, even when they're touching.

SAM operates in "automatic mask generation" mode: it proposes many candidate segments across the image and filters them by quality. Key parameters you'll see in the code include `points_per_side` (density of the sampling grid), `pred_iou_thresh` (confidence threshold), and `stability_score_thresh` (mask stability).

## RLE encoding (Run-Length Encoding)

A segmentation mask is just a 2D array of 0s and 1s, the same dimensions as the original image. Storing thousands of these pixel-by-pixel would be enormous. **RLE** compresses them efficiently: instead of storing `0,0,0,0,0,1,1,1,0,0`, it stores "five 0s, three 1s, two 0s" — just the *run lengths*.

In our code, masks are stored as RLE-encoded dictionaries using `pycocotools` (the COCO dataset's standard library). You'll see `maskUtils.encode()` and `maskUtils.decode()` converting between numpy arrays and RLE format. The RLE masks live in our JSON output files alongside metadata like area, bounding box, and confidence scores.

## CLIP and contrastive vision-language models

You likely know that deep learning models learn representations (embeddings) of their inputs. **CLIP** (Contrastive Language-Image Pre-training, by OpenAI) is a model trained on millions of image-text pairs from the internet. It learned to map images and text descriptions into the *same* embedding space, so that an image of a dog and the text "a photo of a dog" end up near each other.

This is powerful because it means you can classify images using text descriptions alone — no labeled training set needed. You just compare an image's embedding to the embeddings of candidate text labels and pick the closest match.

## Zero-shot classification

**Zero-shot** means classifying into categories the model has never explicitly been trained on. With CLIP, the process is:

1. Encode the image into a 512-dimensional vector
2. Encode each candidate text label (e.g., "a photo of Acropora coral", "a photo of Porites coral") into vectors
3. Compute cosine similarity between the image vector and each label vector
4. The label with the highest similarity wins

This is exactly how our classification pipeline works — we take each cropped coral and compare it against text descriptions of all 20 target genera. No coral-specific training data is needed.

## The three model backends

Our pipeline supports three vision-language models, each with different strengths:

- **BioCLIP-FT** ( `finetuned-bioclip` ): A ViT-B/16 CLIP model fine-tuned on biology/ecology data by the ReefNet project. 512-dimensional embeddings. This is our primary model — best overall for coral classification.
- **BioCLIP2** ( `bioclip2` ): A larger ViT-H/14 model, also biology-focused. Higher capacity but slower.
- **SigLIP2** ( `siglip2` ): Google's Sigmoid Loss Image-Language Pre-training 2 (SO400M). Uses sigmoid loss instead of CLIP's softmax, which can be better at distinguishing similar categories.

All three produce embeddings that map coral crops into a shared image-text space. BioCLIP-FT ( `finetuned-bioclip` ) is the primary model and its embeddings also serve as reusable features for any downstream analysis or future classifier training.

## DINOv2 features

**DINOv2** (by Meta) is a self-supervised vision model that produces rich visual features without any text pairing. Unlike CLIP, it's purely visual — it learned image structure by predicting masked patches of images.

In our segmentation notebook, we extract DINOv2 features from each coral crop as an additional representation. Two variants are used: `dinov2_vits14` (384-dim) and `DinoVdeau-large` (1024-dim). These features capture texture, shape, and color patterns that complement the CLIP embeddings.

## Embeddings as a concept

If you haven't worked with embeddings before: they're fixed-length numerical vectors that capture the "meaning" of an input. Two corals that look similar will have embeddings that are close together (high cosine similarity). The key insight is that different models

capture different notions of similarity — CLIP embeddings reflect how well an image matches a text description, while DINOv2 embeddings reflect pure visual similarity.

# Notebook 1 — CoralMasking.ipynb

Open in Colab

This notebook handles the **segmentation stage**: turning raw reef photos into individual coral crops with metadata.

## What it does

For each reef photo, the notebook: detects every coral colony using the CoralSCOP model (a fine-tuned SAM), draws a precise pixel-level mask around each one, crops out each coral, extracts DINOv2 visual features from each crop, and saves everything (masks, crops, metadata, features) to Google Drive.

## Input

Raw underwater reef photographs (JPEG/PNG) stored in a Google Drive folder. The notebook processes all images in the specified source directory.

## Key code walkthrough

**Setup and model loading** — The notebook mounts Google Drive, installs dependencies (`segment-anything`, `pycocotools`, `torch`), then loads the CoralSCOP model weights. The model is a ViT-B (Vision Transformer, Base size) SAM variant. You'll see the `SamAutomaticMaskGenerator` class, which handles the "find all objects" logic.

**Mask generation** — For each image, `SamAutomaticMaskGenerator` runs a grid of point prompts across the image (controlled by `points_per_side`). Each point may produce a candidate mask. The generator filters these by:

- `pred_iou_thresh` — predicted intersection-over-union (a standard segmentation quality metric: how much the predicted mask overlaps with the model's internal estimate of the "true" mask). Higher = stricter.
- `stability_score_thresh` — how stable the mask is when the decision threshold is varied slightly. Unstable masks likely have ambiguous boundaries.
- `min_mask_region_area` — minimum number of pixels. Tiny fragments are discarded.

**Quality filters** — After raw mask generation, the code applies additional filters before saving. These are critical for ensuring the classification stage receives clean inputs:

- `MIN_AREA = 25,000 px²` — Masks smaller than this are likely rubble or partial colonies, not worth classifying.
- `MAX_ASPECT_RATIO = 3.0` — Very elongated shapes are usually segmentation errors (a strip of reef edge, not a colony).
- `CONTAINMENT_THRESH = 0.80` — If one mask is 80%+ contained within another, the smaller one is considered a duplicate and removed.
- `SHARPNESS_THRESHOLD = 80` — Blurry crops (measured via Laplacian variance) are flagged. Blurriness often means the photo is out of focus or the coral was at the edge of the frame.

**DINOv2 feature extraction** — Each cropped coral is passed through DINOv2 to produce a feature vector. Two model sizes are used: `dinov2_vits14` (Small, 384-dim) and `DinoVdeau-large` (Large, 1024-dim). These features are saved alongside each crop for potential use in downstream analysis.

**RLE mask storage** — Each mask is encoded to RLE format using `pycocotools.mask.encode()` and stored in a JSON file with its bounding box, area, confidence scores, and image source info.

### Output

For each input image, the notebook produces: a JSON file containing all RLE-encoded masks with metadata, cropped coral images (one per detected colony), and DINOv2 feature vectors saved as `.npy` files. These outputs feed directly into the classification notebook.

## Notebook 2 — WeakClassification.ipynb

Open in Colab

This notebook handles **classification**: taking the coral crops from the segmentation stage and assigning each one a genus label using zero-shot CLIP-based models. No labeled training data or expert review is required to run it.

### How it works

The notebook compares each coral crop's visual embedding against text descriptions of 20 Red Sea hard coral genera. Whichever genus description is most similar (highest cosine similarity) becomes the prediction. This is pure zero-shot classification — the model was never trained on coral images.

## Key configurable controls

At the top of the notebook, several variables control behavior:

- `MODEL_VERSION` — Which CLIP backend to use: `finetuned-bioclip` (default, recommended), `bioclip2`, or `siglip2`.
- `PROMPT_FORMAT` — How genera are described to the model: `taxonomy` (Linnaean strings), `descriptive` (morphological descriptions), `common` (common names), `ensemble` (average of all), or `hybrid` (custom per-genus selection).
- `USE_MASK` / `MASK_BACKGROUND` — Whether to use the segmentation mask to isolate the coral or include surrounding reef context.
- `SAMPLE_ONLY` — Set to `True` for a quick test run on a subset.
- `BATCH_SIZE` — Number of crops processed together (default 64).

## Prompt dictionaries

The notebook defines several dictionaries mapping each genus to text descriptions. These are what CLIP compares against each coral crop:

- `genera_taxonomy` — Full Linnaean classification strings (e.g., "Animalia Cnidaria Anthozoa Scleractinia Acroporidae Acropora")
- `genera_descriptive` — Morphological descriptions highlighting visual features
- `genera_common` — Common names plus genus name
- `genera_hybrid` — Custom per-genus prompt selection for best results

The `ensemble` format averages embeddings from multiple prompt formats (3-way for open_clip models, 2-way for SigLIP which skips taxonomy). Prompt engineering is one of the highest-impact areas for improving accuracy — better text descriptions lead to better classifications.

## The 20 target genera

Platygyra, Acropora, Stylophora, Millepora, Favites, Dipsastraea, Favia, Echinopora, Hydnophora, Porites, Acanthastrea, Cyphastrea, Goniastrea, Seriatopora, Pocillopora, Montipora, Leptastrea, Galaxea, Lobophyllia, Pavona — all hard corals found in the Red Sea.

## Main inference loop

The core cell processes all crops in batches. For each crop it: loads and preprocesses the image, computes the CLIP image embedding, calculates cosine similarity against all pre-encoded genus prompts, applies softmax to get a confidence score, and records the top prediction. Progress is logged as it runs.

### Output

The notebook produces a CSV file named `coral_weak_labels_{model}_{prompt}_{crop_mode}_{sample_tag}.csv` with one row per coral crop: image name, coral ID, predicted genus, confidence score, and the full probability distribution across all genera.

### Visualization and evaluation

After classification, the notebook includes cells for: per-genus grids showing top-confidence predictions (a quick visual sanity check), random sample views grouped by predicted genus, and comparison against a small expert-labeled test set (78 crops reviewed by a marine biologist) to estimate real-world accuracy.

## Data and folder structure

All data lives on Google Drive under two main folders. Here's what's where:

### Coral_Segmentation_Outputs (segmentation results)

This folder contains everything produced by the CoralMasking notebook:

- `reef_masks/` — JSON files with RLE-encoded masks for each processed reef image. One JSON per source image, each containing all detected coral masks with bounding boxes, areas, and quality scores.
- `coral_crops/` — Individual coral images cropped from the reef photos using the masks. These are the inputs to the classification stage.
- `features/` — DINOv2 feature vectors ( `.npy` files) extracted from each crop.
- `metadata/` — Processing logs and summary statistics.

### Coral_Classification (classification results)

This folder contains the classification pipeline's inputs and outputs:

- `genera_taxonomy.json` — Text descriptions for each genus used as CLIP comparison targets.
- `coral_weak_labels_*.csv` — Classification output: predicted genus, confidence score, and probability distributions for each crop.
- `checkpoints/` — Processing checkpoints for resuming interrupted runs.
- `ground_truth_genus_counts.csv` — Reference genus distribution from expert survey, used for evaluation.

### Notebooks

The four project notebooks also live on Google Drive (accessible via Colab):

- **CoralMasking.ipynb** — Segmentation (your main notebook #1)
- **WeakClassification.ipynb** — Zero-shot classification (your main notebook #2). Open in Colab
- **CoralClassification_v2_Pipeline.ipynb** — An extended pipeline that adds tiered auto-labeling, expert review, and a supervised classifier on top of the zero-shot approach. Under development — not required for the core workflow.
- **Millepora_Analysis.ipynb** — A focused analysis of Millepora (fire coral) classification accuracy. Supplementary — read it when you need to understand how we audit per-genus precision.

## Running the pipeline — step by step

### Prerequisites

Both notebooks run in **Google Colab** with a GPU runtime. Before starting, make sure you have access to the shared Google Drive folders and that you can open the notebooks in Colab.

### Step 1: Segmentation (CoralMasking.ipynb)

1. Open the notebook in Colab. Set the runtime to **GPU** (Runtime → Change runtime type → T4 GPU).
2. Run the setup cells to mount Google Drive and install dependencies.
3. Set the **source image folder** path to your reef photos on Drive.
4. Set the **output folder** path where masks and crops will be saved.
5. Run the mask generation cells. For a large batch, this can take hours — the notebook will print progress.
6. Check the quality filter summary: it will report how many masks were kept vs. filtered and why (too small, too elongated, contained within another, blurry).
7. The DINOv2 feature extraction runs after masking. This adds time but only needs to run once per batch.

### Step 2: Classification (WeakClassification.ipynb)

1. Open the notebook in Colab with GPU runtime.
2. Set `MODEL_VERSION` to `finetuned-bioclip` (or experiment with other backends).
3. Choose a `PROMPT_FORMAT` — start with `ensemble` for best results.

4. Point the input path to the crops folder from Step 1.
5. Run all cells. The notebook processes each crop through the CLIP model and produces a CSV with genus predictions and confidence scores.
6. Use the built-in visualization cells to spot-check results: per-genus grids of top-confidence predictions give a quick visual sanity check.

### Common gotchas

- **Colab timeouts** — Free Colab sessions disconnect after ~90 minutes of inactivity. Keep the browser tab active, and rely on the checkpoint system to recover.
- **Drive mounting** — If `drive.mount()` fails, try re-authenticating. Colab sometimes loses the Drive connection after a runtime restart.
- **GPU memory** — If you hit CUDA out-of-memory errors, reduce the batch size or restart the runtime to clear the GPU.
- **Checkpoint corruption** — If a checkpoint file seems corrupt (error on loading), delete it and restart that batch. The notebook is idempotent for already-processed images.

## Current state and next steps

### What's been done

**Segmentation** is operational. CoralMasking.ipynb can process reef photos end-to-end: SAM segmentation, quality filtering, crop extraction, and DINOv2 feature extraction all work.

**Zero-shot classification** has been run on the current dataset using WeakClassification.ipynb. Results from the latest run: 14,224 total coral crops processed across 20 hard coral genera. Mean zero-shot confidence across all crops is 0.485. Top-confidence predictions are visually plausible, but genus-level accuracy varies — the Millepora audit (below) shows why visual spot-checks are essential.

**Millepora precision audit** has been completed: among 129 sharp, high-confidence Millepora predictions, only 58.9% were correct (66 yes, 46 no, 17 uncertain). This highlights that even confident predictions can have high false-positive rates for certain genera — an important caveat to keep in mind.

### What's in progress

**Improving classification accuracy** is the main focus. Different combinations of CLIP model backends and prompt formats yield different results — systematic evaluation against the 78-crop expert test set helps identify which configurations work best for each genus.

## Open questions and areas for improvement

- **Genera taxonomy tuning** — The text descriptions in `genera_taxonomy` directly affect zero-shot accuracy. Better descriptions = better initial classification. This is an area where domain knowledge can make a big impact.
- **Millepora and other problem genera** — Some genera are intrinsically hard to distinguish from photos. Millepora is a known problem case; the audit showed 41.1% false positives. Strategies like additional text prompts, few-shot examples, or genus-specific classifiers could help.
- **Scale** — As we process more reef photos, the pipeline's throughput and checkpointing will need attention. Colab's session limits are a practical constraint for large batches.

## Getting started

Your best first move: open CoralMasking.ipynb and WeakClassification.ipynb side by side. Read through each one with this guide as a companion, running cells on a small test batch (a handful of reef photos) to see the outputs at each stage. The concepts in the *Key concepts* section above will click much faster once you see the masks, crops, and classification scores for yourself.